

线性表在民航系统中的应用：实验报告

姓名：张启元

学号：2024201541

2025 年 11 月 8 日

1 问题分析

本问题是一个航班数据管理系统，需要处理大量的航班信息数据。通过抽象成数据结构问题，可以将航班信息封装为结构体，利用向量、链表等数据结构进行存储和操作。编程时需要完成以下内容：

- 设计航班信息的抽象数据类型（ADT）
- 实现数据文件的读取和解析
- 实现数据的排序、查询、插入和删除功能
- 设计高效的查找算法（如二分查找）
- 实现素数判断、鞍点检测等特殊功能
- 构建用户交互界面
- 实现航班暂停恢复机制
- 解决最多乘机航线问题

2 数据结构设计与实现

2.1 ADT 设计

- **数据对象**：航班信息，包含航班 ID、日期、类型、编号、机场信息、时间信息、飞机信息、费用信息等

- **数据关系：**航班数据之间通过向量和链表组织，支持顺序访问和条件查询
- **基本操作：**
 - 数据读取和初始化
 - 数据排序
 - 条件查询
 - 数据插入和删除
 - 素数判断
 - 鞍点检测
 - 航班暂停与恢复
 - 最长航线查找

2.2 ADT 的物理实现

航班数据的物理实现采用结构体封装，关键代码片段如下：

```
struct Flight{  
    ll FlightID;           // 航班ID  
    ll DepartureDate;      // 启航日期  
    bool IsIntl;           // 航班类型（国际1/国内0）  
    string FlightNo;       // 航班编号  
    ll DepartureAirport;   // 启航机场ID  
    ll ArrivalAirport;     // 抵达机场ID  
    double DepartureTime;  // 启航时间  
    double ArrivalTime;   // 抵达时间  
    ll AirplaneID;         // 飞机ID  
    int AirplaneModel;     // 飞机机型  
    ll AirFares;           // 基础费用  
    ll PeakSeasonRates;    // 旺季价格
```

```

    ll OffSeasonRates;           //淡季价格
    bool IsSuspended = false;   //是否暂停
};

```

链表节点结构用于组织特定条件下的航班数据：

```

struct AirplaneFlight{
    ll FlightID;
    double DepartureTime;
    double ArrivalTime;
    ll DepartureAirport;
    ll ArrivalAirport;
    AirplaneFlight *next = nullptr;
};

```

删除数据管理结构：

```

struct PointedAirplaneModel{
    Flight F;
    PointedAirplaneModel *next = nullptr;
};

```

3 算法设计与实现

3.1 算法设计

本系统采用多种算法解决不同问题：

1. **埃氏筛法**：用于生成素数表，快速判断航班费用是否为素数
2. **快速排序**：对航班数据进行多关键字排序（按启航机场、抵达机场、航程时长）
3. **二分查找**：在有序数据中快速定位特定机场对的航班

4. **鞍点检测算法**：在价格矩阵中查找鞍点航班
5. **链表操作**：管理删除的航班数据
6. **记忆化搜索**：解决最多乘机问题，找到所有最长航线
7. **批量数据处理**：支持按多种条件批量删除航班数据

表 1: 算法复杂度分析

算法	时间复杂度	空间复杂度
素数筛法	$O(n \log \log n)$	$O(n)$
快速排序	$O(n \log n)$	$O(\log n)$
二分查找	$O(\log n)$	$O(1)$
鞍点检测	$O(m \times n)$	$O(1)$
最多乘机问题	$O(n^2)$	$O(n)$
批量删除	$O(n)$	$O(k)$

3.2 算法实现

素数筛法关键代码：

```
void SelectPrimeNumber(){
    PrimeNumberTable[0] = 1;
    PrimeNumberTable[1] = 1;
    ll t = 2;
    while (t < 15000){
        ll j = 2;
        if (PrimeNumberTable[t] == 0){
            while (t * j < 15000){
                PrimeNumberTable[t * j] = 1;
                j++;
            }
        }
        t++;
    }
}
```

```

        }
    }
    t++;
}
}

```

二分查找实现:

```

pair<ll, ll> BinarySearch(vector<Flight> &f, const ll &DepartureID,
    const ll &ArrivalID){
    if (f.empty()) return {-1, -1};

    // 查找左边界 (第一个满足条件的)
    int left = 0, right = f.size() - 1;
    int first_pos = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        // 由于是降序, 比较逻辑需要调整
        if (f[mid].DepartureAirport > DepartureID) {
            left = mid + 1;
        } else if (f[mid].DepartureAirport < DepartureID) {
            right = mid - 1;
        } else {
            // DepartureAirport 相等, 比较 ArrivalAirport
            if (f[mid].ArrivalAirport > ArrivalID) {
                left = mid + 1;
            } else if (f[mid].ArrivalAirport < ArrivalID) {
                right = mid - 1;
            }
        }
    }
}

```

```

        } else {
            // 找到匹配, 向左找第一个
            first_pos = mid;
            right = mid - 1;
        }
    }
}

if (first_pos == -1) return {-1, -1};
// ... 右边界查找代码
return {first_pos, last_pos + 1};
}

```

最多乘机问题的记忆化搜索:

```

vector<vector<ll>> findAllLongestPaths(const vector<Flight>& flights,
    ll currentFlightID, const Flight& currentFlight) {
    if (memo.count(currentFlightID)) {
        return memo[currentFlightID];
    }

    vector<vector<ll>> allPaths = {{currentFlightID}};
    int maxLength = 1;

    for (const auto& flight : flights) {
        if (flight.DepartureTime >= currentFlight.ArrivalTime +
            0.020833 &&!flight.IsSuspended
            && flight.DepartureAirport == currentFlight.
                ArrivalAirport) {

```

```

vector<vector<ll>> subPaths = findAllLongestPaths(flights
    , flight.FlightID, flight);

// 如果找到更长的路径, 重置所有路径
if (!subPaths.empty() && subPaths[0].size() + 1 >
    maxLength) {
    maxLength = subPaths[0].size() + 1;
    allPaths.clear();
    for (const auto& path : subPaths) {
        vector<ll> newPath = {currentFlightID};
        newPath.insert(newPath.end(), path.begin(), path.
            end());
        allPaths.push_back(newPath);
    }
}

// 如果路径长度相同, 添加到结果中
else if (!subPaths.empty() && subPaths[0].size() + 1 ==
    maxLength) {
    for (const auto& path : subPaths) {
        vector<ll> newPath = {currentFlightID};
        newPath.insert(newPath.end(), path.begin(), path.
            end());
        allPaths.push_back(newPath);
    }
}
}
}

```

```
memo[currentFlightID] = allPaths;  
return allPaths;  
}
```

3.3 关键算法特点

1. **排序算法**：采用多关键字排序，优先按启航机场降序，其次按抵达机场降序，最后按航程时长升序
2. **鞍点检测**：在三维价格空间（基础费用、旺季价格、淡季价格）中查找鞍点
3. **链表管理**：使用链表保存删除的数据，支持后续的排序和查询操作
4. **用户交互**：采用分层菜单设计，提供友好的命令行界面
5. **错误处理**：对用户输入进行验证，防止非法输入导致程序崩溃

4 实验环境

- **硬件环境**：Intel Core i9 CPU，16GB 内存
- **软件环境**：Windows 操作系统，C++11 及以上版本，Visual Studio Code
- **编译环境**：支持 C++11 标准的编译器
- **数据文件**：CSV 格式的航班数据文件

5 实验结果与分析

程序成功实现了所有预定功能，包括：

- **数据管理**：成功读取和解析 CSV 文件，处理超过 10000 条航班记录
- **排序功能**：多关键字排序正确，排序时间在可接受范围内

- **查询功能：**
 - 素数费用查询：利用预计算的素数表实现快速查询
 - 鞍点检测：正确识别价格矩阵中的鞍点航班
 - 执飞顺序：按时间顺序展示飞机的航班安排
- **数据操作：**
 - 批量增加：从文件批量添加新航班数据
 - 批量删除：支持按机场、航班类型、飞机型号等多种条件删除
 - 链表管理：成功将删除数据组织成链表并进行后续操作
- **高级功能：**
 - 航班暂停恢复：动态管理航班的可用状态
 - 最多乘机问题：使用记忆化搜索找到所有最长航线

5.1 性能分析

通过计时功能观察到：

- **数据初始化：**在毫秒级别完成
- **数据排序：**对于大规模数据，排序时间在合理范围内
- **查询操作：**二分查找显著提高了查询效率
- **最多乘机问题：**记忆化搜索避免了重复计算，提高了算法效率

5.2 系统优势

1. **模块化设计：**各功能模块独立，便于维护和扩展
2. **算法高效：**采用合适的算法和数据结构，保证系统性能

3. **用户友好**：分层菜单和输入验证提供良好的用户体验
4. **健壮性强**：完善的错误处理机制
5. **功能完整**：覆盖了航班管理的各个方面

程序运行稳定，各功能模块响应迅速，算法效率满足实际应用需求。系统具有良好的扩展性，可以方便地添加新的功能模块。

6 总结与展望

本次实验成功实现了一个完整的航班数据管理系统，通过合理的数据结构设计和算法选择，解决了航班数据管理中的多个关键问题。系统不仅具备基本的数据操作功能，还实现了多个高级特性，如鞍点检测、最长航线查找等。

未来可以进一步优化的方向包括：

- 引入数据库技术进行数据持久化存储
- 实现图形用户界面（GUI）提升用户体验
- 添加更多数据分析和统计功能
- 优化算法性能，支持更大规模的数据处理